CS 61A
Past Exam Practice

Gleam and Linked Lists - Solutions

Solutions last updated: Sunday, November 1, 2026

Your Name: _____

Your Student ID: _____

Room you are taking the exam in: _____

Name and SID of the person to your **left**: _____

Name and SID of the person to your **right**: _____

You have 110 minutes. There are 7 questions of varying credit. (38 points total)

Question	1	2	3	4	5	6	7	Total
Points	6	6	4	6	5	6	5	38

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◎ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ (Don't do this)

Make sure to **write your student ID at the top of each page** in the provided blank.

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam. I will not use any other unauthorized materials or devices during this exam. I will not communicate with anyone other than the proctors during this exam. I will not share any information about the exam with anyone until after the end of the exam period.

Acknowledge that you have read and agree to the honor code above and **sign your name below**:

_____

This content is protected and may not be shared, uploaded, or distributed.

Q1 *Beadazzled* **(6 points)**

Original source: Spring 2022 Midterm 2 - Question 4 | **Level:** Regular

Implement `make_necklace`, which creates a linked list whose values repeat the strings in `beads` in order until the result has `length` elements. Assume `beads` is non-empty and `length` is non-negative.

```
def make_necklace(beads: list[str], length: int) -> LinkedList[str]:
    """Return a linked list of LENGTH beads, repeating BEADS in order.

    >>> wavy = make_necklace(["~", "@"], 3)
    >>> print(wavy)
    (~ @ ~)
    >>> wavy
    Link(first='~', rest=Link(first='@', rest=Link(first='~', rest=())))
    >>> print(make_necklace(["}", "O", "{"], 9))
    (} O { } O { } O {)
    >>> make_necklace(["x"], 0) == ()
    True
    """
    if _______:                          # (a)
        return _______                   # (b)
    return Link(
        _______,                         # (c)
        make_necklace(_______, _______) # (d), (e)
    )
```

Q1.1 (1 point) Fill in blank (a).

```
length == 0
```

Q1.2 (1 point) Fill in blank (b).

● `()` ○ `None` ○ `Link()` ○ `[]`

Q1.3 (1 point) Fill in blank (c).

```
beads[0]
```

Q1.4 (2 points) Fill in blank (d).

```
beads[1:] + beads[:1]
```

Q1.5 (1 point) Fill in blank (e).

```
length - 1
```

Gleam Reference

This page summarizes the Gleam features used in this worksheet. Types are capitalized, function bodies end with the expression they return, and values are immutable.

Functions and types

```
pub fn square(x: Int) -> Int {
  x * x
}

pub fn twice(f: fn(a) -> a, x: a) -> a {
  f(f(x))
}
```

`a` is a type variable. For example, `List(a)` means a list whose elements all have the same type, whatever that type is.

Anonymous functions use `fn`:

```
fn(x) { x + 1 }
```

Define named recursive helpers at the top level, not inside another function. A recursive call is tail recursive when its result is returned directly, with no remaining operation.

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `%`
- Comparison: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Boolean: `&&`, `||`, `!`

Case expressions and patterns

```
case n {
  0 -> "zero"
  _ if n < 0 -> "negative"
  _ -> "positive"
}
```

A `case` can match multiple subjects: `case x, y { ... }`. The `_` pattern matches any value, and `if` adds a guard to a pattern.

Lists

```
case s {
  [] -> []
  [first, ..rest] -> [first, ..rest]
  [a, b, ..tail] -> [a, b, ..tail]
}
```

- `[]` is empty.
- `[first, ..rest]` splits a non-empty list.
- `[first, ..rest]` also constructs a new list.
- `#(x, y)` is a two-element tuple.

Useful standard-library functions:

- `list.append(xs, ys)` joins two lists.
- `list.map(xs, f)` applies `f` to each value.
- `list.filter(xs, keep)` keeps values satisfying `keep`.
- `list.length(xs)` returns a list's length.
- `int.absolute_value(n)` returns an integer's absolute value.

Import their modules with `import gleam/list` or `import gleam/int`.

Q2 *Beadazzled, the Gleam-quel* **(6 points)**

Adapted from: Spring 2022 Final - Question 11 | **Level:** Challenge

Implement `make_necklace`, which returns a Gleam list whose values repeat the items in `beads` in order until the result has `length` elements. Assume `beads` is non-empty and `length` is non-negative.

`list.append(xs, ys)` returns a new list containing the elements of `xs` followed by the elements of `ys`.

```
1  import gleam/list
2
3  pub fn make_necklace(beads: List(a), length: Int) -> List(a) {
4    case length, beads {
5      _______ -> _______                          // (a), (b)
6      _, [first, ..rest] -> [
7        _______,                                  // (c)
8        ..make_necklace(_______, _______)         // (d), (e)
9      ]
10     _, [] -> []
11   }
12 }
13
14 assert make_necklace(["~", "*"], 3) == ["~", "*", "~"]
15 assert make_necklace(["~", "^"], 4) == ["~", "^", "~", "^"]
16 assert make_necklace([">", "0", "<"], 9)
17   == [">", "0", "<", ">", "0", "<", ">", "0", "<"]
```

Q2.1 (2 points) Fill in blanks (a) and (b).

(a) `0, _`

(b) `[]`

Q2.2 (1 point) Fill in blank (c).

```
first
```

Q2.3 (2 points) Fill in blank (d).

```
list.append(rest, [first])
```

Q2.4 (1 point) Fill in blank (e).

```
length - 1
```

Q3 *Listquality* **(4 points)**

Adapted from: Past exam practice - Treequality | **Level:** Regular

Part A: `all`. Implement `all`, which returns `True` when every Boolean in `s` is `True`, including when `s` is empty, and returns `False` otherwise.

```
1  pub fn all(s: List(Bool)) -> Bool {
2    case s {
3      [] -> _______                                  // (a)
4      [first, ..rest] -> _______ && all(_______)     // (b), (c)
5    }
6  }
7
8  assert all([True, True, True]) == True
9  assert all([True, False, True]) == False
10 assert all([]) == True
```

Q3.1 (1 point) Fill in blank (a).

- (●) `True`
- ( ) `[]`
- ( ) `False`
- ( ) `Nil`

Q3.2 (1 point) Fill in blanks (b) and (c).

(b) `first`

(c) `rest`

Part B: `zip`. Implement `zip`, which combines two lists into a list of two-element tuples. If the inputs have different lengths, stop when the shorter list runs out. The type variables may differ, so the two input lists do not need to contain the same type.

```
1  pub fn zip(s0: List(a), s1: List(b)) -> List(#(a, b)) {
2    case s0, s1 {
3      [], _ -> []
4      _, [] -> []
5      [first0, ..rest0], [first1, ..rest1] -> [
6        _______,                                     // (d)
7        ..zip(_______, _______)                      // (e), (f)
8      ]
9    }
10 }
11
12 assert zip([1, 2, 3], [-1, -2, -3])
13   == [#(1, -1), #(2, -2), #(3, -3)]
14 assert zip(["a", "b"], [1, 2, 3]) == [#("a", 1), #("b", 2)]
```

Q3.3 (1 point) Fill in blank (d).

```
#(first0, first1)
```

Q3.4 (1 point) Fill in blanks (e) and (f).

(e) `rest0`

(f) `rest1`

Q4 *Split Decision* **(6 points)**

Adapted from: Summer 2026 - 6B - `split` | **Level:** Regular

Implement `split` tail recursively. It takes a list `s` and a predicate `pred`, then returns a tuple containing two lists: items that satisfy `pred` and items that do not. Each result list is in reverse order because items are added to the front of an accumulator.

```
pub fn split(
  s: List(a),
  pred: fn(a) -> Bool,
) -> #(List(a), List(a)) {
  split_tail(_______)                              // (a)
}

fn split_tail(
  s: List(a),
  pred: fn(a) -> Bool,
  satisfy: List(a),
  not_satisfy: List(a),
) -> #(List(a), List(a)) {
  case s {
    [] -> _______                                  // (b)
    [first, ..rest] ->
      case pred(first) {
        True -> split_tail(_______)                // (c)
        False -> split_tail(_______)               // (d)
      }
  }
}

assert split([], fn(x) { x % 2 == 0 }) == #([], [])
assert split([1, 2, 3, 4, 5], fn(x) { x % 2 == 0 })
  == #([4, 2], [5, 3, 1])
```

Q4.1 (1 point) Fill in blank (a).

```
s, pred, [], []
```

Q4.2 (1 point) Fill in blank (b).

```
#(satisfy, not_satisfy)
```

Q4.3 (2 points) Fill in blank (c).

```
rest, pred, [first, ..satisfy], not_satisfy
```

Q4.4 (2 points) Fill in blank (d).

```
rest, pred, satisfy, [first, ..not_satisfy]
```

Q5 *Rotation Situation* **(5 points)**

Adapted from: Summer 2026 - 6B - Question 7 | **Level:** Regular

Implement `rotate`, which shifts all elements of a list `lst` left by `k` positions. Assume `k` is non-negative. The function should still work when `k` is greater than the length of the list.

`list.append(xs, ys)` returns the elements of `xs` followed by the elements of `ys`.

```
1  import gleam/list
2
3  pub fn rotate(lst: List(a), k: Int) -> List(a) {
4    case k, lst {
5      0, _ -> _______                                  // (a)
6      _, [] -> []
7      _, [first, ..rest] ->
8        rotate(
9          _______(_______, _______),                  // (b), (c), (d)
10         _______,                                    // (e)
11       )
12   }
13 }
14
15 assert rotate([1, 2, 3, 4, 5], 0) == [1, 2, 3, 4, 5]
16 assert rotate([1, 2, 3, 4, 5], 1) == [2, 3, 4, 5, 1]
17 assert rotate([1, 2, 3, 4, 5], 2) == [3, 4, 5, 1, 2]
18 assert rotate([1, 2, 3, 4, 5], 7) == [3, 4, 5, 1, 2]
```

Q5.1 (1 point) Fill in blank (a).

```
lst
```

Q5.2 (1 point) Fill in blank (b).

○ `list.filter` ● `list.append` ○ `list.length`

Q5.3 (2 points) Fill in blanks (c) and (d).

(c) `rest`

(d) `[first]`

Q5.4 (1 point) Fill in blank (e).

```
k - 1
```

Q6 *Don't Skip This* **(6 points)**

Adapted from: Fall 2023 - Final 7 - `part` | **Level:** Challenge

Definition. A *skip-partition* of a positive integer `n` is a list of positive integers in increasing order that sums to `n` and does not contain any duplicates or consecutive numbers.

Implement `part`, which takes positive integers `n` and `m`. It returns a list of all skip-partitions of `n` whose elements are all greater than or equal to `m`. The provided `start_with` function may be used in the implementation.

```
1  import gleam/list
2
3  fn start_with(first: Int) -> fn(List(Int)) -> List(Int) {
4    fn(rest) { [first, ..rest] }
5  }
6
7  pub fn part(n: Int, m: Int) -> List(List(Int)) {
8    case m {
9      _ if m == n -> _______                          // (a)
10     _ if m > n -> []
11     _ ->
12       list.append(
13         list.map(_______, _______),                 // (b), (c)
14         part(n, m + 1),
15       )
16   }
17 }
18
19 assert part(12, 2)
20   == [[2, 4, 6], [2, 10], [3, 9], [4, 8], [5, 7], [12]]
21 assert part(10, 1)
22   == [[1, 3, 6], [1, 9], [2, 8], [3, 7], [4, 6], [10]]
```

Q6.1 (2 points) Fill in blank (a).

```
[[m]]
```

Q6.2 (2 points) Fill in blank (b).

```
part(n - m, m + 2)
```

Q6.3 (2 points) Fill in blank (c).

```
start_with(m)
```

Q7 *After Party* **(5 points)**

Adapted from: Fall 2023 - Midterm 2 6 - `after` | **Level:** Challenge

Implement `after`, which takes a list `s` and values `a` and `b`. It returns whether an element of `s` equal to `b` appears after an element of `s` equal to `a`.

```
1  pub fn after(s: List(t), a: t, b: t) -> Bool {
2    find(s, a, fn(rest) { _______ })                // (a)
3  }
4
5  fn find(s: List(t), n: t, f: fn(List(t)) -> Bool) -> Bool {
6    case s {
7      [] -> _______                                 // (b)
8      [first, ..rest] if first == n -> f(rest)
9      [_, ..rest] -> find(rest, n, f)
10   }
11 }
12
13 assert after([3, 6, 5, 4], 6, 4) == True
14 assert after([3, 6, 5, 4], 4, 6) == False
15 assert after([3, 6, 5, 4], 6, 6) == False
```

Q7.1 (4 points) Fill in blank (a). You may not use `case`, `&&`, `||`, `[`, or `]`.

```
find(rest, b, fn(_) { True })
```

Q7.2 (1 point) Fill in blank (b).

```
False
```